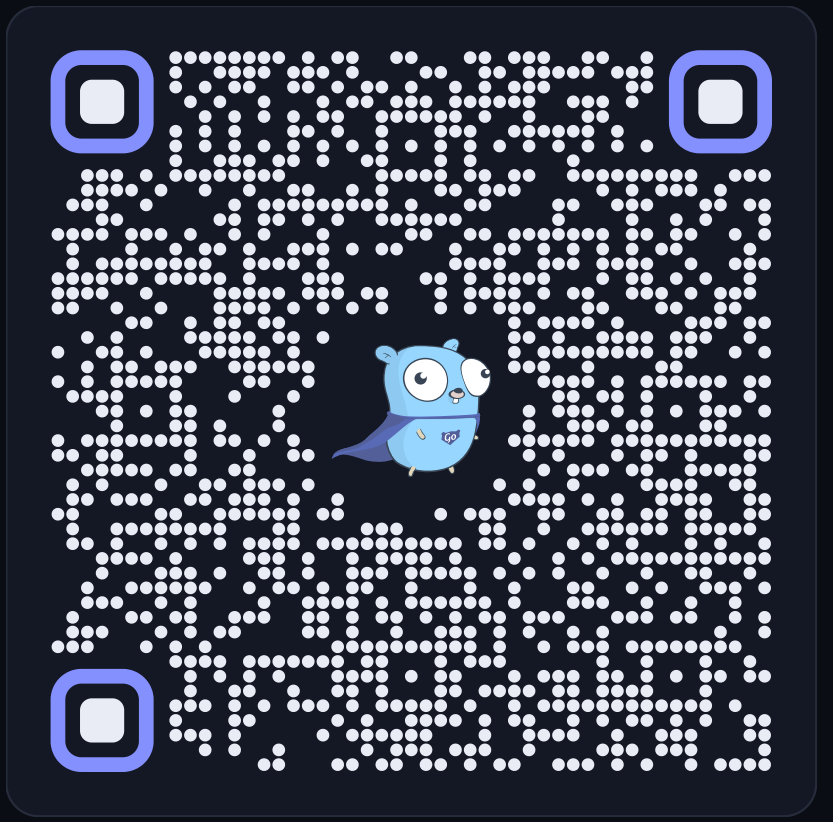
REDOWAN DELOWAR

Context propagation in Go

Lessons from operating large-scale distributed systems at DoorDash (Wolt)



Scan to get the slides

https://rednafi.com/appearances

$ whoami

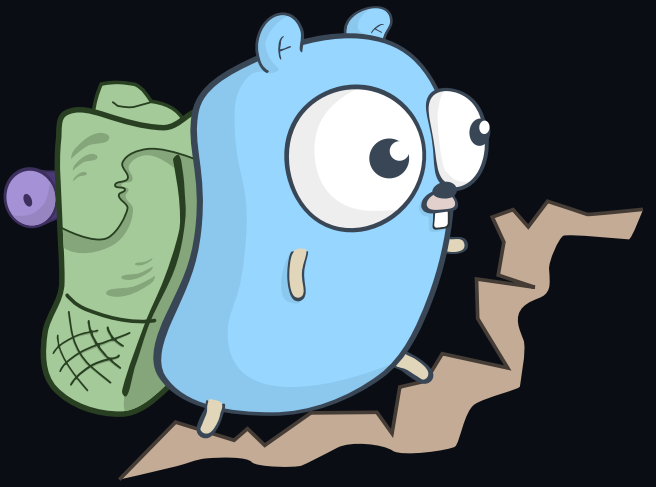
Hi, I'm Redowan, @rednafi on the web.

I oscillate between product and platform work at DoorDash/Wolt.

Interested in databases, streaming, o11y, and failure modes of large-scale distsys.

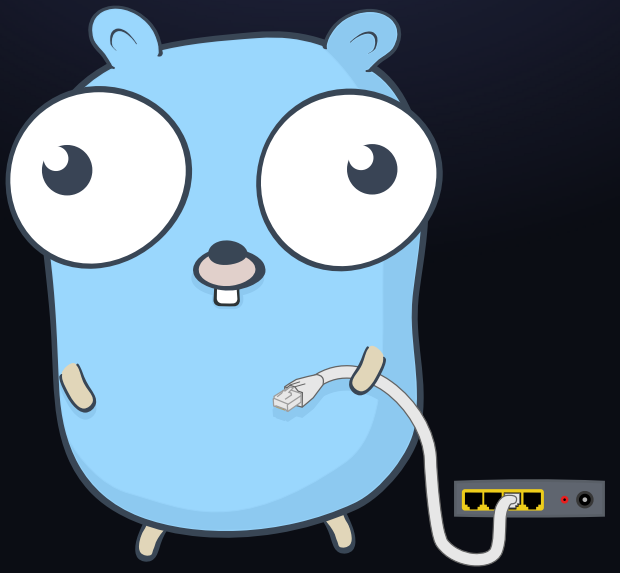
Writing at rednafi.com.

github.com/rednafi
linkedin.com/in/rednafi



> A distributed system is one in which the failure of a computer you didn't even know existed can render your own computer unusable.

— LESLIE LAMPORT, 1987



THE PROBLEM

The log says a context stopped, but it hides the reason

order-service

```
ERROR place order: context canceled
ERROR fetch menu: context deadline exceeded
ERROR place order: context canceled
ERROR charge card: context canceled
ERROR fetch menu: context deadline exceeded
ERROR place order: context canceled
```

ANY OF THESE CAN CAUSE IT

- The client hung up
- A deadline set by a caller ran out
- The server started to shut down
- Code called `cancel()`

The cancel often comes from another service or from a library.



“

I had a case when on production I got random "context canceled" log messages. And in the case like that you don't even know where to dig and how to investigate it further.

— BULLGARE, GOLANG/GO#51365, 2022



Overview

01 Context basics

02 Passing a context down

03 Recording why a context stopped

04 Context values

05 Enforcing the rules with linters

01

PART 1

Context basics

The work continues after the client leaves



The query holds a database connection the whole time

The inventory and payment services do work that nobody uses

Go can't stop a goroutine from the outside

- A goroutine has no kill method
- So you send the work a signal, and the work stops itself
- All goroutines for one request need the same signal
- The signal also needs a time limit



> When a request is canceled or times out, all the goroutines working on that request should exit quickly so the system can reclaim any resources they are using.

Sameer Ajmani, Go blog, 2014

context.Context

> Package context defines the Context type, which carries deadlines, cancellation signals, and other request-scoped values across API boundaries and between processes.

context package doc

```
type Context interface {
    Deadline() (deadline time.Time, ok bool)   <-- when to give up
    Done() <-chan struct{}                     <-- closes when the work should stop
    Err() error                                <-- Canceled or DeadlineExceeded
    Value(key any) any                         <-- data about the request
}
```

Only the code that creates a context can cancel it

A context is read-only. To add a deadline or a value, make a child context

Each With function makes a child, so contexts form a tree

`Background()`

The empty root context

`WithCancel(parent)`

Stop when the code calls cancel

`WithTimeout(parent, d)`

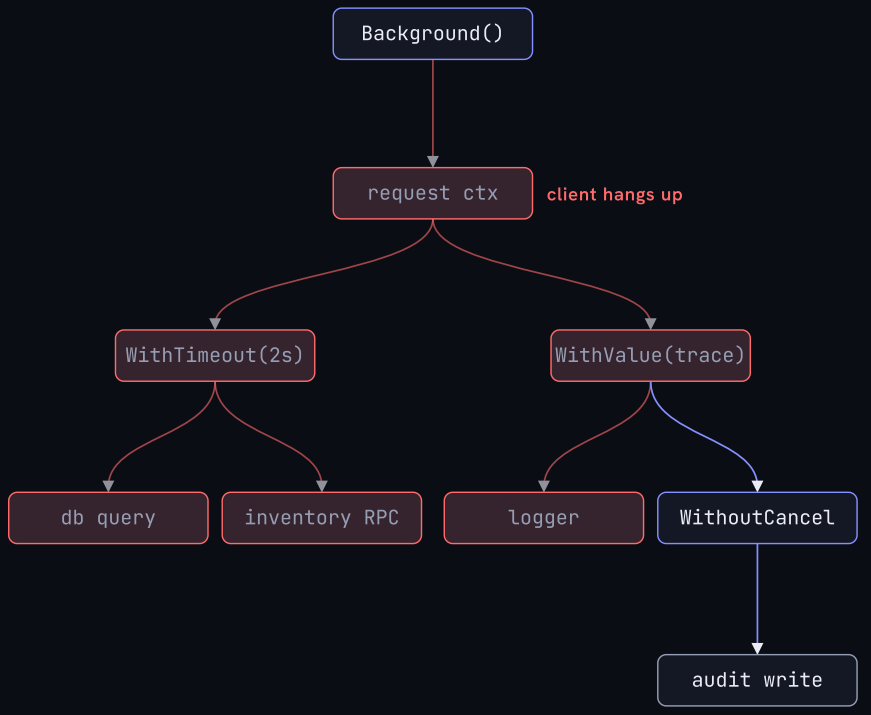
Stop after a duration. `WithDeadline` stops at a time

`WithValue(parent, k, v)`

Add one value

`WithoutCancel(parent)`

Keep the values and drop the stop signal (Go 1.21)



When the client hangs up, all work below the request stops, except the WithoutCancel branch.

A context travels inside a process and between processes



In-process: your code passes `ctx` from function to function.
Distributed: gRPC and OpenTelemetry copy it into headers. They can copy only what your code passed down.

Each layer passes ctx to the next

```go
// Do: pass ctx down through every layer.
func (h *Handler) GetOrder(w http.ResponseWriter, r *http.Request) {
    ctx, cancel := context.WithTimeout(r.Context(), 2*time.Second)
    defer cancel()
    order, err := h.orders.Get(ctx, r.PathValue("id"))
    // ...
}

func (s *OrderService) Get(ctx context.Context, id string) (Order, error) {
    return s.store.Find(ctx, id)
}

func (s *OrderStore) Find(ctx context.Context, id string) (Order, error) {
    row := s.db.QueryRowContext(ctx, findOrderSQL, id)
    // ...
}
```

the handler sets the budget

the service passes it on

the driver waits on ctx.Done()

A cancel stops only the code that checks the context

```go
// Don't: ignore ctx in the loop.
func resizeAll(ctx context.Context, images []Image) error {
    for _, img := range images {
        resize(img) // 200ms each
    }
    return nil
}

// Do: check ctx before each image.
func resizeAll(ctx context.Context, images []Image) error {
    for _, img := range images {
        if err := ctx.Err(); err != nil {
            return err
        }
        resize(img)
    }
    return nil
}
```

keeps going after a cancel

stops before the next image

`cancel()` only closes the `Done` channel

Make every loop and every wait check `ctx`, or pass it to code that checks it

The standard library stops its own work when the context ends

```go
// Do: pass ctx to standard library calls.
ctx, cancel := context.WithTimeout(ctx, 3*time.Second)
defer cancel()

req, err := http.NewRequestWithContext(ctx, http.MethodGet, url, nil)
resp, err := http.DefaultClient.Do(req)

rows, err := db.QueryContext(ctx, listOrdersSQL, userID)

out, err := exec.CommandContext(ctx, "pg_dump", dbName).Output()
```

closes the connection

the driver gets ctx and stops the query

kills the process

The HTTP server cancels `r.Context()` when the client disconnects

`signal.NotifyContext` cancels a context when a signal such as `SIGTERM` arrives

Go has documented these rules since 2014

Jul 2014

Go blog

Go Concurrency Patterns: Context. Pass ctx as the first argument.

Aug 2016

Go 1.7

context moves into the standard library.

Feb 2021

Go blog

Contexts and structs. Don't store a context in a struct.

Nov 2022

Google style guide

Public rules for contexts and custom contexts.

2023

Go 1.20, 1.21

Causes and WithoutCancel.

Most context bugs I see are in these groups

1

Work that runs too long

The client left, but the query continues.

2

Work that stops too early

The audit write fails after the response.

3

A stop with an unknown reason

The log says only context canceled.

4

A value that goes missing

The user ID lookup returns nil.

02

PART 2

Passing a context down

PASSING A CONTEXT DOWN

Pass ctx as the first argument to each function

```
// Don't: start a new context here.
return s.store.Find(context.Background(), id)
```

loses the caller's deadline

```
// Do: pass the caller's context.
return s.store.Find(ctx, id)
```

Use `context.Background()` only in `main` and `init`

In tests, use `t.Context()` (Go 1.24)

Use `context.TODO()` where no context is passed in yet

Never pass a `nil` context

PASSING A CONTEXT DOWN

Pass ctx to each call, and don't store it in a struct

```go
// Don't: keep the context in a struct.
type Notifier struct {
    ctx    context.Context
    client *sms.Client
}


// Do: pass the context to each call.
func (n *Notifier) Send(ctx context.Context, msg string) error {
    return n.client.Send(ctx, msg)
}
```

one deadline for every call

> When you store the context in a struct, you obscure lifetime to the callers, or worse intermingle two scopes together in unpredictable ways.

Go blog, Contexts and structs, 2021

PASSING A CONTEXT DOWN

Wait on the timer and ctx.Done() together

```
// Don't: sleep without a check for cancel.
time.Sleep(backoff(attempt))   <-- a cancel can't stop it


// Do: stop the wait when ctx is done.
select {
case <-time.After(backoff(attempt)):
case <-ctx.Done():             <-- a cancel ends the wait
    return ctx.Err()
}
```

Every blocking wait needs a `ctx.Done()` case, including channel sends

Since Go 1.23, a `time.After` timer that nobody reads is garbage collected

PASSING A CONTEXT DOWN

Give background work its own context and timeout

```
// Don't: use the request context for background work.
go h.audit.Write(r.Context(), orderID)        <-- fails after the response

// Do: keep the values and set a new timeout.
detached := context.WithoutCancel(r.Context())   <-- Go 1.21
h.pool.Submit(func() {
    ctx, cancel := context.WithTimeout(detached, 5*time.Second)   <-- its own timeout
    defer cancel()
    h.audit.Write(ctx, orderID)
})
```

`context.Background()` also works, but it loses the trace ID and the log fields

Use a pool, so shutdown can wait for the work

PASSING A CONTEXT DOWN

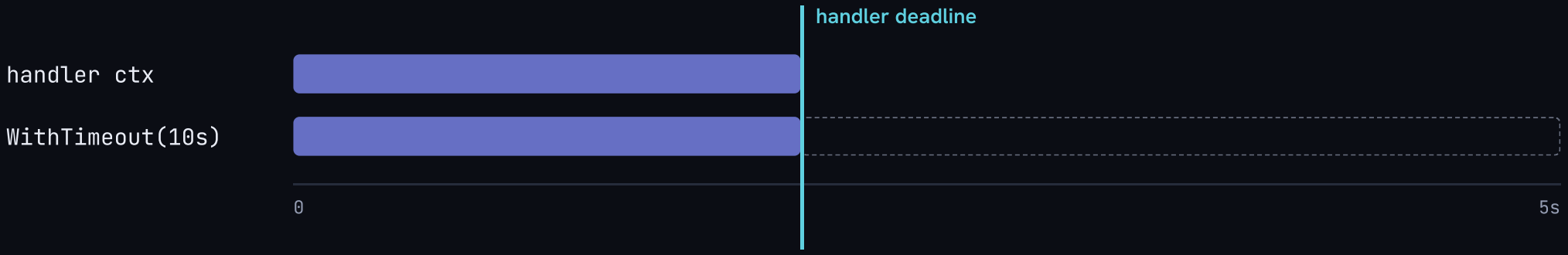
Set the deadline at the edge, because a child can't extend it

```go
// Don't: ask for more time deeper down.
ctx, cancel := context.WithTimeout(ctx, 10*time.Second)

// Do: set the deadline once, in the handler.
ctx, cancel := context.WithTimeout(r.Context(), 2*time.Second)
defer cancel()
```

still ends at the handler's 2s

the edge sets the budget



The 10s the child asked for, cut at 2s

Give an inner call its own timeout only when it needs less time than the request.

03

PART 3

Recording why a context stopped

RECORDING WHY A CONTEXT STOPPED

Cancel with a cause that says why

```go
// Do: give each cancel a reason.
func processOrder(ctx context.Context, id string) error {
    ctx, cancel := context.WithCancelCause(ctx)
    defer cancel(errors.New("processOrder: completed"))

    if err := checkInventory(ctx, id); err != nil {
        cancel(fmt.Errorf("inventory %s: %w", id, err))
        return err
    }
    // ...
}
```

cancel takes a cause (Go 1.20)

default reason

specific reason

```
$ context.Cause(ctx)
inventory ord-123: connection refused
```

The first cancel wins, and later calls do nothing

`ctx.Err()` still returns `context.Canceled`

`context.Cause` works on any context below this one

RECORDING WHY A CONTEXT STOPPED

WithTimeoutCause sets a cause only when the timer fires

```
// Don't: expect this cause on every stop.
ctx, cancel := context.WithTimeoutCause(ctx, 5*time.Second,
    fmt.Errorf("processOrder %s: 5s timeout", id)) <-- used only on timeout
defer cancel() <-- cancel() sets no cause
```

PATH	CONTEXT.CAUSE(CTX)
The timer fires	`processOrder ord-123: 5s timeout`
The function returns first	`context canceled`
An error path calls `cancel()`	`context canceled`

RECORDING WHY A CONTEXT STOPPED

Put a cancel cause on top of the timeout

“

> If users need to set the cause on both the timer and cancel() paths, they can stack contexts.

Sameer Ajmani, golang/go#56661, 2022

```go
// Do: stack a cancel cause on the timeout.
func processOrder(ctx context.Context, id string) error {
    ctx, cancelCause := context.WithCancelCause(ctx)
    ctx, cancelTimeout := context.WithTimeoutCause(ctx, 5*time.Second,
        fmt.Errorf("processOrder %s: 5s timeout", id))
    defer cancelTimeout()
    defer cancelCause(errors.New("processOrder: completed"))

    // call cancelCause(...) on each error path
}
```

takes a cause on every path

has the timer and the deadline

deferred last, so it runs first

`ctx.Err()` is `DeadlineExceeded` when the timer fires

`ctx.Deadline()` still works, so gRPC can send it on

RECORDING WHY A CONTEXT STOPPED

Wrap DeadlineExceeded in the cause, so errors.Is still matches

```
// Don't: use a cause that drops DeadlineExceeded.
ctx, cancel := context.WithTimeoutCause(ctx, 20*time.Millisecond,
    errors.New("inventory: budget spent")) <-- errors.New drops DeadlineExceeded
defer cancel()

_, err := http.DefaultClient.Do(req.WithContext(ctx)) <-- since Go 1.23, err is the cause
errors.Is(err, context.DeadlineExceeded) <-- false


// Do: wrap DeadlineExceeded with %w.
ctx, cancel := context.WithTimeoutCause(ctx, 20*time.Millisecond,
    fmt.Errorf("inventory: budget spent: %w", context.DeadlineExceeded)) <-- errors.Is now returns true
```

RECORDING WHY A CONTEXT STOPPED

Log the error and the cause as separate fields

```go
// Do: log the error and the cause.
if err := ctx.Err(); err != nil {
    logger.LogAttrs(ctx, slog.LevelError, "request failed",
        slog.Any("err", err),
        slog.Any("cause", context.Cause(ctx)),
    )
}
```

```
level=ERROR msg="request failed"
    err="context deadline exceeded"
    cause="processOrder ord-123: 5s timeout: context deadline exceeded"
```

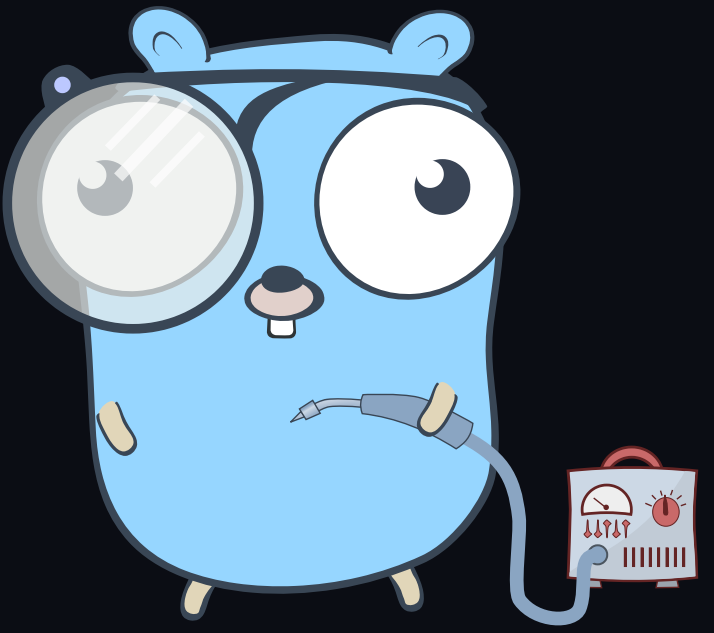
`err` gives the type of failure, so dashboards can group by it

`cause` gives the reason, so the on-call engineer knows what to fix

RECORDING WHY A CONTEXT STOPPED

Give every cancel in your code a cause

- The `net/http` server cancels `r.Context()` with no cause when the client hangs up
- Many client libraries are older than Go 1.20
- `os/signal.NotifyContext` got a cause only in Go 1.26
- The cause stays in the process. gRPC sends only the deadline
- If every cancel in your code has a cause, then a bare `context canceled` came from outside your code



04

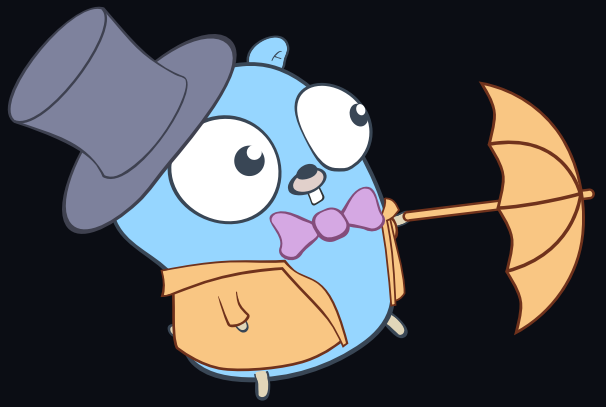
PART 4

Context values

Put only optional values in the context

FINE IN A CONTEXT

- Trace, request, and correlation IDs
- Auth data set by middleware
- Request-scoped log fields
- Idempotency keys



MAKE IT A PARAMETER

- User data the function works on
- Database handles and service clients
- Configuration
- Business inputs

> Use context Values only for request-scoped data that transits processes and APIs, not for passing optional parameters to functions.

context package doc

Use an unexported key type, so keys can't collide

```
// Don't: use a string as the key.
ctx = context.WithValue(ctx, "user", "alice")     // package auth
ctx = context.WithValue(ctx, "user", "audit-bot") // package audit
ctx.Value("user")                                 // "audit-bot" ← alice is gone


// Do: use an unexported key type.
package auth

type ctxKey struct{} ← only this package can build one
```

`Value` goes up the tree and returns the value of the first equal key

Two string keys with the same text are equal

CONTEXT VALUES

Hide the key behind a setter and a getter

```go
// Do: keep the key behind two functions.
package auth

func WithUser(ctx context.Context, u User) context.Context {
    return context.WithValue(ctx, ctxKey{}, u)
}

func UserFromContext(ctx context.Context) (User, bool) {
    u, ok := ctx.Value(ctxKey{}).(User)
    return u, ok
}
```

the only way to set it

the only type assertion

Callers get a typed value and a bool. The key stays private

A missing value returns `false`

CONTEXT VALUES

Don't hide a dependency in a context

```go
// Don't: find the transaction in the context.
func (s *BookStore) Create(ctx context.Context, b Book) (int64, error) {
    var db DBTX = s.db
    if tx, ok := TxFromContext(ctx); ok {
        db = tx
    }
    // ...
}

// Do: pass the transaction as an argument.
err := store.Tx(ctx, func(tx book.Store) error {
    // use tx for every write
})
```

false if a caller forgets WithTx

The writes then go to the pool, outside the transaction

The signature hides the dependency, so the compiler can’t find this bug

05

PART 5

Enforcing the rules

ENFORCING THE RULES

Let linters catch what code review misses

`go vet: lostcancel`	A cancel function that is never called
`staticcheck: SA1012`	A `nil` context passed to a function
`staticcheck: SA1029`	A built-in type, such as a string, used as a key
`containedctx`	A context stored in a struct field
`contextcheck`	A function that uses a context it didn't get from its caller
`errorlint`	`err == context.Canceled`, which fails on a wrapped cause

ENFORCING THE RULES

Wrap the constructors so every cancel needs a cause

```
// Do: make the cancel func take a cause.
package xctx

func WithCancel(parent context.Context) (context.Context, context.CancelCauseFunc) {
    ctx, cancel := context.WithCancelCause(parent)
    return ctx, func(cause error) {                                  <- the cancel func needs a reason
        cancel(fmt.Errorf("%w: %w", cause, context.Canceled))        <- errors.Is still matches
    }
}


# Do: block the plain constructors.
# .golangci.yml
linters:
  settings:
    forbidigo:
      forbid:
        - pattern: ^context\.With(Cancel|Timeout|Deadline)$
          msg: use xctx, which takes a cause
```

At DoorDash, Alex Hornbake built this as `xctx` for a Go monorepo that many teams share.

WRAP-UP

The checklist I use in code review

1. Pass `ctx` first to each function, and don't store it in a struct
2. Wait on `ctx.Done()` in every loop and every blocking wait
3. Set the deadline at the edge, and give background work its own timeout
4. Give every cancel a cause, including the normal return
5. Make every cause wrap `context.Canceled` or `context.DeadlineExceeded`
6. Log `ctx.Err()` and `context.Cause(ctx)` as separate fields
7. Keep values optional and keys unexported, behind a setter and a getter
8. Let linters check the rest

REFERENCES

Further reading

GO BLOG

- Go Concurrency Patterns: Context, Sameer Ajmani, 2014
- Go Concurrency Patterns: Pipelines and cancellation, Sameer Ajmani, 2014
- Contexts and structs, Jean Barkhuysen and Matt T. Proud, 2021

MY POSTS

- What canceled my Go context?
- Avoiding collisions in Go context keys
- What belongs in Go's context values?
- Is passing user ID through context an antipattern?

GO ISSUES

- #26356: context: ease debugging of where a context was canceled?
- #40221: context: add WithoutCancel
- #51365: context: add APIs for writing and reading cancelation cause
- #56661: context: add APIs for setting a cancelation cause when deadline or timer expires

DOCS AND STYLE GUIDES

- Package context
- Go 1.20 release notes: context
- Go 1.21 release notes: context
- Google Go style guide: Contexts
- Google Go style guide: Custom contexts

LINTERS

- go vet: lostcancel
- staticcheck: SA1012, SA1029
- golangci-lint: containedctx, contextcheck, errorlint, forbidigo

Thanks

Questions?

REDNAFI.COM

